

M08 / STUDY COMPANION

M08 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	5
How to revise	5
DE08-01 Ingestion architecture: source -> landing -> staging	5
60-second recall	5
Recall task	5
DE08-02 Defensive CSV ingestion	5
60-second recall	5
Recall task	5
DE08-03 JSON and nested-record ingestion	6
60-second recall	6
Recall task	6
DE08-04 Parquet ingestion and metadata inspection	6
60-second recall	6
Recall task	6
DE08-05 REST API contracts, status codes and timeouts	6
60-second recall	6
Recall task	6
DE08-06 API authentication, pagination and rate limits	7
60-second recall	7
Recall task	7
DE08-07 Retries, backoff, jitter and transient failures	7

60-second recall	7
Recall task	7
DE08-08 Database extraction and safe query boundaries	7
60-second recall	7
Recall task	8
DE08-09 Historical batch landing and raw metadata	8
60-second recall	8
Recall task	8
DE08-10 Incremental loads and high-water marks	8
60-second recall	8
Recall task	8
DE08-11 CDC concepts and change-event semantics	8
60-second recall	8
Recall task	9
DE08-12 Schema drift, compatibility, time zones and quarantine	9
60-second recall	9
Recall task	9
DE08-13 Checkpoint/state management and replay safety	9
60-second recall	9
Recall task	9
DE08-14 Idempotent landing and content identity	10
60-second recall	10
Recall task	10

DE08-15 | Late-arriving data, event time and processing time 10

60-second recall 10

Recall task 10

DE08-16 | Ingestion observability: counts, latency and run metadata 10

60-second recall 10

Recall task 10

DE08-17 | Ingestion integration, reconciliation and backfill proof 11

60-second recall 11

Recall task 11

Read this first

QUICK REVISION BOOK

Use this only after you have studied the learner book. Each lesson keeps the canonical language and core proof, plus one recall task. This is not a replacement for hands-on practice.

Coverage: 17 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to revise

Read the lesson card, close the page, explain it in your own words, then answer the recall task. If recall is weak, return to that lesson only - not the entire module.

DE08-01 | Ingestion architecture: source -> landing -> staging

60-second recall

Strong evidence separates immutable raw landing from rebuildable staging, keeps quarantine with reasons, and commits state only after complete success. A retry creates new run evidence rather than overwriting yesterday. "Landing proves what arrived. Staging proves what my current contract accepted. Quarantine proves what it rejected and why. State proves how far the repeatable process is allowed to resume.

[Refresh source: DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)

Recall task

A vendor sends the same filename orders.csv every day. Design a landing convention that proves which bytes belonged to Monday and Tuesday even though the source filename is identical. Explain why renaming the newest file in place is weaker than a run-scoped immutable landing copy.

DE08-02 | Defensive CSV ingestion

60-second recall

Good CSV: 4 source rows -> 4 accepted, 0 quarantine. Bad fixture must preserve raw bytes and classify defects; accepted+quarantine must equal source rows. Leading-zero IDs stay text.

[Refresh source: Python csv module documentation](#)

Recall task

Create a new CSV with a quoted comma in a product description, one duplicate order_id, and one leading-zero customer ID. Predict what a naïve line.split(',') parser would do. Then use the CSV parser correctly and prove the identifier's leading zeros survive.

DE08-03 | JSON and nested-record ingestion

60-second recall

J3001 produces exactly two item-grain rows. State the key (order_id,line_no) for this fixture and reconcile item totals from source versus flattened rows.

[Refresh source: Python json module documentation](#)

Recall task

Add an order J3003 with an empty items array and a customer object missing name. Decide what belongs in an order-header table, what belongs in item staging, and what (if anything) should be quarantined. Do not solve by forcing a fake item row unless the destination contract explicitly requires one.

Work the grain by hand before coding For J3001, order grain gives one header with two items. Item grain gives (J3001,1) and (J3001,2). J3002 gives one item row.

DE08-04 | Parquet ingestion and metadata inspection

60-second recall

Parquet logical row count and keys match source, schema is inspected, projected read contains only requested columns, and metadata row-group evidence is saved. Native pyarrow execution is required on learner PC.

[Refresh source: Apache Arrow - Reading and Writing Parquet](#)

Recall task

Write a second Parquet file with a deliberately different row-group size. Compare metadata and explain what changed physically while the logical table remains the same.

Read the metadata like evidence After writing the tiny file with row_group_size=2, four source rows should normally produce two row groups. Record what your actual PyArrow version reports rather than copying that expectation blindly.

DE08-05 | REST API contracts, status codes and timeouts

60-second recall

/health=200; unauthenticated order endpoint=401; authenticated valid call=200; missing path=404. JSON decode is never used as proof of HTTP success and timeout is explicit.

[Refresh source: Requests Quickstart](#)

Recall task

Imagine an endpoint returns status 200 with body {"status":"error","items":[]}. Decide whether HTTP success alone permits checkpoint advancement. Write the additional application-level contract you would need.

Build a response decision table

Before coding a generic handler, write what the client should do. 200 + expected contract -> continue. 200 + wrong body shape -> contract failure, do not checkpoint as clean success. 401 -> authentication/configuration failure, stop and repair credentials.

DE08-06 | API authentication, pagination and rate limits

60-second recall

page_size=3 yields all 10 unique v1 IDs; one page-2 429 is retried according to Retry-After; 401 is not repeatedly retried; token is absent from saved logs.

[Refresh source: Requests authentication documentation](#)

Recall task

Set page size 4. Predict page count and sizes before execution. Then change only the pagination contract in a small mock response so the next cursor is a string token rather than a page number. Explain why production code should isolate pagination mechanics instead of scattering page += 1 through transformation logic.

Prove completeness, not just page count With ten records and page_size=3, the expected IDs are A4001 through A4010.

DE08-07 | Retries, backoff, jitter and transient failures

60-second recall

One-time 500 recovers under bounded retry. 429 may retry. Wrong auth fails quickly. Attempt count and backoff are visible. Write retries are not assumed safe without idempotency semantics.

[Refresh source: urllib3 Retry reference](#)

Recall task

You now receive 503 three times and then 200. Maximum attempts is 3 total, not 3 retries after the first call. Predict whether success is reachable under that definition, then make the attempt-count wording explicit in code and logs.

Make attempt semantics unambiguous Teams often say "three retries" when one person means three total calls and another means one original plus three retries. The course code uses max_attempts: the total number of calls including the first.

DE08-08 | Database extraction and safe query boundaries

60-second recall

After (08:15,D5002) the local extract returns D5003 then D5004. Query uses explicit columns, parameters and deterministic order. Read-only connection rejects mutation.

[Refresh source: PostgreSQL 18 - COPY](#)

Recall task

The source gains a column `internal_notes`. Your extraction should remain unchanged because it names explicit fields. Prove it, then explain why schema drift in unused source columns may be non-breaking while drift in a consumed field may be breaking.

Prove the source stayed unchanged

Before extraction, record source row count and optionally file/database hash for the local fixture. After the read-only query, verify the same control.

DE08-09 | Historical batch landing and raw metadata

60-second recall

Both historical files land in one named run, each manifest entry records original name/bytes/SHA256, and a second run does not overwrite first-run evidence.

[Refresh source: Microsoft - Data lake architecture](#)

Recall task

The second file is resent with one corrected byte but the same name. Show how content hash reveals a new version. Decide whether to keep both landed copies and what lineage fields staging needs to identify which one was used.

Historical ingestion needs a receipt Treat the manifest like a receiving log. It tells a future engineer what objects were acquired in one run before interpretation. For the two supplied files, record both original basenames, byte lengths and independent hashes.

DE08-10 | Incremental loads and high-water marks

60-second recall

Initial v1 ingest returns 10 unique records and final state (2026-08-05T09:20:00Z,A4010). A4007/A4008 tie proves timestamp-only watermarks unsafe. No-new-data rerun returns zero.

[Refresh source: DataTalksClub - Docker/PostgreSQL data-engineering workshop](#)

Recall task

Use a fresh fixture where three IDs share the same timestamp and the run intentionally stops after the second tied ID. Persist (timestamp, second_id) and prove the third ID is returned on resume. Then show what a timestamp-only watermark would do.

Walk the tied timestamp slowly The sequence contains A4007 at 09:00 and A4008 at 09:00. Suppose a run commits state after A4007 due to a controlled batch boundary.

DE08-11 | CDC concepts and change-event semantics

60-second recall

Applying ordered CDC events leaves A5001 status paid and removes A5002 under current-state semantics while preserving event history. Duplicate event ID does not duplicate effect.

[Refresh source: Binod Suman Academy - Change Data Capture \(CDC\) | Why & How | Use case | System Design](#)

Recall task

Receive an update event before its create because files arrived out of order. Decide whether to quarantine, buffer, reconstruct from snapshot or allow upsert. Explain what evidence you need rather than inventing a universal rule.

Separate event log from current-state projection

The CDC fixture contains four events but final current state contains only A5001. That is not data loss: the event log preserves all four events, while the current-state projection applies create/update/delete semantics.

DE08-12 | Schema drift, compatibility, time zones and quarantine

60-second recall

v2 raw page lands but contributes zero v1 staging rows; records are quarantined as unsupported/incompatible and live v1 checkpoint does not advance. Offset-less time is not silently assigned local time.

[Refresh source: DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)

Recall task

A v3 payload adds discount_pct but keeps all required v1 fields identical. Decide whether this is compatible for the v1 consumer and what metadata you would record so the ignored new field is not invisible to governance.

Compatibility belongs to the consumer contract

A producer can truthfully announce "schema v2" while your v1 consumer still cannot safely use it. Compatibility is therefore relative to a consumer and its expectations.

DE08-13 | Checkpoint/state management and replay safety

60-second recall

Replay creates new landing evidence but live checkpoint remains byte-identical under no-commit policy; failure before commit leaves prior state intact; checkpoint ownership is explicit.

Recall task

Create two independent ingestion jobs that accidentally share one checkpoint path. Explain the race/ownership problem and redesign state paths so each source/contract has a clear owner.

State is a commit record, not a scratch note Take a hash of the live checkpoint before replay. Run replay with no-commit state. Hash it again. Equality is a simple proof that replay did not alter live resume position.

DE08-14 | Idempotent landing and content identity

60-second recall

Two landings can keep identical bytes/run evidence while logical staging remains one row per business key. SHA proves byte identity, not business identity.

[Refresh source: PyCon US 2025 - Building Resilient Data Pipelines: The Power of Idempotency](#)

Recall task

Create two files with different byte hashes but the same order IDs and values because column order differs. Explain why byte-level deduplication would keep both while business-key staging can still avoid duplicate logical rows.

Use the right identity at the right layer Suppose two landing runs contain byte-identical orders.csv. Their SHA-256 values match, proving content identity. Their run IDs differ, proving separate acquisition attempts. Staging keyed by order_id can still contain one logical row/order.

DE08-15 | Late-arriving data, event time and processing time

60-second recall

L2 is late under 60-minute policy; event time is preserved. Classification changes predictably under a new tolerance and does not rewrite the business timestamp.

[Refresh source: Apache Beam - event time and processing time concepts](#)

Recall task

Change tolerance to 15 minutes. Predict which classifications change before running. Explain why a watermark/tolerance change can alter operational state and may require deliberate reprocessing rather than silently applying a new rule midstream.

Calculate lateness explicitly For L2, event_time is 09:10Z and arrival is 10:25Z: lag = 75 minutes, therefore late under a 60-minute tolerance. L1 has 5 minutes. L3 uses +05:30; convert both event and arrival to UTC before subtraction.

DE08-16 | Ingestion observability: counts, latency and run metadata

60-second recall

Run metadata includes counts/retries/state/status and accepted+quarantined reconciles to source scope. Missing row with SUCCESS is detected by equation, not hidden by logs.

[Refresh source: OpenTelemetry - Logs](#)

Recall task

A run acquires 10 records, stages 9 and has 0 quarantine. Your status says SUCCESS. Use the count equation to detect silent loss and redefine the run status/alert condition.

Turn metrics into controls A useful run record can answer: Did acquisition complete? How many source records were seen? How many were accepted or rejected? How many retries happened?

What checkpoint existed before and after? Which raw content hashes belong to this run?

DE08-17 | Ingestion integration, reconciliation and backfill proof

60-second recall

Integration proof includes immutable landing, contract validation, quarantine, bounded retry, composite state, replay/idempotency, late/event-time reasoning, structured run metadata, failure injection and handoff.

Recall task

The package includes a different assessment fixture with IDs/timestamps changed. Complete it without reopening the first solution. Mastery means you can reconstruct the reasoning - source contract, identity, state, validation and replay - when the values change.